



Subject Name-WEB DESIGNING

Course Code:-BECS25-103

UNIT-C

Cascading Style Sheets



UNIT- C TOPICS- 2

➤ CSS Advanced

- Grouping
- Dimension
- Display
- Positioning
- Floating
- Align
- Pseudo class

- Navigation Bar

- Image Sprites

- Attribute sector

- CSS Color

- Creating page Layout and Site Designs.



GROUPING

- **Grouping in CSS** allows the application of the same style declarations to multiple selectors simultaneously.
- This practice reduces **code repetition**, improves **readability**, and can potentially enhance page load times by minimizing the overall size of the stylesheet.

To group selectors, separate each selector with a **comma (,)** within the CSS rule.



GROUPING

Syntax:

Code

```
selector1, selector2, selector3 {  
  property: value;  
  /* Other declarations */  
}
```

Example:

To apply the same `color` and `font-family` to `h1`, `h2`, and `p` elements:

Code

```
h1, h2, p {  
  color: blue;  
  font-family: Arial, sans-serif;  
}
```

This rule will style all `h1`, `h2`, and `p` elements on the page with a blue color and the specified font family.



GROUPING

BENEFITS OF GROUPING:

- **Reduced Code:** Avoids writing the same style rules multiple times for different elements.
- **Improved Readability:** Makes the CSS easier to understand and maintain.
- **Efficiency:** Can lead to smaller file sizes and potentially faster rendering, as the browser processes fewer distinct style rules.



DIMENSION

CSS dimension properties allow you to control the size of HTML elements, including their width, height, and the space around them. These properties are fundamental for creating layouts and ensuring your design is responsive and visually appealing across different screen sizes.

- To set the dimensions of an element, use [width](#) and [height](#).
- Alternatively, use [max-height](#), [min-width](#), [line-height](#), and others.



DIMENSION

Key CSS Dimension Properties:

- **width:** Sets the horizontal size of an element.
- **height:** Sets the vertical size of an element.
- **max-width:** Sets the maximum allowed horizontal size for an element, preventing it from exceeding a certain width.
- **min-width:** Sets the minimum allowed horizontal size for an element, preventing it from shrinking below a certain width.
- **max-height:** Sets the maximum allowed vertical size for an element.
- **min-height:** Sets the minimum allowed vertical size for an element.



DIMENSION

Key CSS Dimension Properties:

PROPERTY	DESCRIPTION
<code>height</code>	Sets the height of an element
<code>width</code>	Sets the width of an element
<code>min-height</code>	Sets the minimum height of an element
<code>min-width</code>	Sets the minimum width of an element
<code>max-height</code>	Sets the maximum height of an element
<code>max-width</code>	Sets the maximum width of an element
<code>line-height</code>	Sets the spacing height between lines of text



DIMENSION

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Dimensions Demo</title>

<style>
  body {
    font-family: Arial, sans-serif;
    background-color: #f7f7f7;
    margin: 0;
    padding: 40px;
  }

  h1 {
    text-align: center;
    color: #333;
  }

  .example {
    border: 2px dashed #888;
    background-color: #e0f7fa;
    color: #333;
    padding: 10px;
    margin: 20px auto;
    text-align: center;
  }
```

```
/* Demonstrate width and height */
.box1 {
  width: 200px;
  height: 100px;
  background-color: #4CAF50;
  color: white;
}

/* Demonstrate max-width and min-width */
.box2 {
  width: 50%;
  min-width: 100px;
  max-width: 400px;
  height: 100px;
  background-color: #2196F3;
  color: white;
}

/* Demonstrate max-height and min-height */
.box3 {
  width: 200px;
  min-height: 50px;
  max-height: 150px;
  overflow: auto; /* Allows scroll when content is taller than max-height */
  background-color: #f44336;
  color: white;
}
</style>
```



DIMENSION

```
</head>
<body>

<h1>CSS Dimension Properties Example</h1>

<div class="example box1">
  <strong>width:</strong> 200px;<br>
  <strong>height:</strong> 100px;|
</div>

<div class="example box2">
  <strong>width:</strong> 50%;<br>
  <strong>min-width:</strong> 200px;<br>
  <strong>max-width:</strong> 400px;
</div>

<div class="example box3">
  <strong>min-height:</strong> 50px;<br>
  <strong>max-height:</strong> 150px;<br><br>
  This box has some text inside.
  If the text becomes longer than 150px in height, the box will scroll instead of growing taller.
  <br><br>this is box for showing dimension </div>

</body>
</html>
```

CSS Dimension Properties Example

width: 200px;
height: 100px;

width: 50%;
min-width: 200px;
max-width: 400px;

min-height: 50px;
max-height: 150px;

This box has some text
inside. If the text becomes
longer than 150px in height,
the box will scroll instead of
growing taller.



DISPLAY

- The display property is an important CSS property for **controlling layout**.
- It **specifies whether an HTML element is treated as a block or an inline element**.
- Every HTML element has a default display value, depending on what type of element it is. The default display value for most elements is block or inline.
- The display property is **used to change the default display behavior of HTML elements**.
- The display property in CSS specifies **how an element is displayed in the document flow**. It defines whether an element behaves like a block, inline, flex, grid, or even becomes hidden.



DISPLAY

CSS Display Syntax

The syntax of the `display` property is as follows:

```
display: value;
```

The commonly used values for the `display` property are as follows:

- `inline`: allows an element to behave like an inline-level element
- `block`: allows an element to behave like a block-level element
- `inline-block`: formats the element as inline-level but also allows to set height/width like block-level element
- `flex`: sets the element as a flex container to have a flexible layout of its child elements
- `grid`: sets the element as a grid container to create complex layouts
- `none`: removes the element from the document leaving no space



DISPLAY

CSS Display Inline

The inline value of the display property allows a block-level element to behave like an inline element. The inline elements occupy only the required width for their content.

```
div {  
  border: 2px solid black;  
  margin-bottom: 12px;  
  padding: 4px;  
}  
  
div.after {  
  display: inline;  
}
```

Before display: inline

I am a first div element.

I am a second div element.

After display: inline

I am a first div element.

I am a second div element.

In the above example, the inline value causes the block-level element div to behave like an inline element.

Hence, it takes as much width as required and doesn't start on a new line.



DISPLAY

CSS Display Block

The **block** value of the **display** property causes an inline element to behave like a block element. For example,

```
span {  
  border: 2px solid black;  
  margin-bottom: 12px;  
  padding: 4px;  
}  
  
span.after {  
  display: block;  
}
```

Before display: block

I am a first span element. I am a second span element.

After display: block

I am a first span element.

I am a second span element.

In the example, the block value causes the inline-level element span to behave like a block element. Hence, it starts on a new line and covers the entire width.



DISPLAY

CSS Display Inline Block

```
span {  
  width: 60px;  
  height: 30px;  
  padding: 10px;  
  margin: 20px;  
  
  border: 2px solid black;  
  background-color: greenyellow;  
}  
  
span.after {  
  display: inline-block;  
}
```

Before display: inline-block

In this paragraph, the  element is color in green-yellow color. The width and height don't work, the padding value doesn't push surrounding content away, and the top/bottom margins don't work.

After display: inline-block

In this paragraph, the  element is color in green-yellow color. The width and height work, the padding value push surrounding content away, and the top/bottom margins work.



DISPLAY

CSS Display Flex

The **flex** value of the display property allows for efficient alignment and distribution of the space between the child elements within the parent element.

```
ul.parent {  
  display: flex;  
  background-color: greenyellow;  
  padding: 0px;  
}  
  
li {  
  background: skyblue;  
  border: 1px solid black;  
  padding: 12px;  
  margin: 8px;  
  list-style: none;  
}
```



In the above example, the `flex` value of the `display` property aligns the `li` elements in a horizontal layout.



DISPLAY

CSS Display Flex

The **flex** value of the display property allows for efficient alignment and distribution of the space between the child elements within the parent element.

```
ul.parent {  
  display: flex;  
  background-color: greenyellow;  
  padding: 0px;  
}  
  
li {  
  background: skyblue;  
  border: 1px solid black;  
  padding: 12px;  
  margin: 8px;  
  list-style: none;  
}
```



In the above example, the `flex` value of the `display` property aligns the `li` elements in a horizontal layout.



DISPLAY

CSS Display None

The **none** value of the **display** property is used to hide an element and remove it from the normal flow of the document.

```
h1 {  
  border: 2px solid black;  
  background-color: greenyellow;  
}
```

Heading: CSS Display Property

The display: none hides the element and removes the space from the document.



POSITIONING

The CSS **position** property is used to define the position of an element on a webpage.

The location of the positioned element is set with the four properties: **top, left, right, and bottom**. These properties only work when the position property is set and have different positioning behaviors.

The position property has the following five values:

- static (default value)
- Relative
- Absolute
- Fixed
- sticky



POSITIONING

CSS static Position

The **static** value of the **position** property allows elements to be positioned accordingly to the normal flow of the document. The static position is not affected by the **top, right, bottom, and left** values.

```
p {  
    border: 1px solid black;  
    padding: 4px;  
}  
  
p.main {  
    position: static;  
    top: 50px; /* doesn't work */  
    right: 50px; /* doesn't work */  
    bottom: 50px; /* doesn't work */  
    left: 50%; /* doesn't work */  
    background-color: greenyellow;  
}
```



POSITIONING

CSS static Position

```
<!DOCTYPE html>
<html lang="en">
<head>
<style>
* {
  box-sizing: border-box;
}

.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#two {
  position: static;
  top: 20px;
  left: 20px;
  background: blue;
}
</style>
</head>
<body>
<div class="box" id="one">One</div>
<div class="box" id="two">Two</div>
<div class="box" id="three">Three</div>
<div class="box" id="four">Four</div>
</body>
</html>
```





POSITIONING

CSS Fixed Position

Fixed positioning removes the element from the flow and positions it relative to the viewport. It remains in place even when the page scrolls.

- An element with `position: fixed;` is positioned relative to the viewport, which means **it always stays in the same place even if the page is scrolled**. The `top`, `right`, `bottom`, and `left` properties are used to set the final location of the element.
- A fixed element does not leave a gap in the page where it would normally have been located.
- This `<div>` element has `position: fixed;`



POSITIONING

CSS Fixed Position

```
<!DOCTYPE html>
<html lang="en">
<head>
<style>
* {
  box-sizing: border-box;
}

.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#two {
  position: fixed;
  top: 20px;
  left: 20px;
  background: blue;
}
</style>
</head>
<body>
<div class="box" id="one">One</div>
<div class="box" id="two">Two</div>
<div class="box" id="three">Three</div>
<div class="box" id="four">Four</div>
</body>
</html>
```



POSITIONING

CSS relative Position

- The element is positioned according to the normal flow of the document, and then offset *relative to itself* based on the **values of top, right, bottom, and left**.
- The offset does not affect the position of any other elements; thus, the space given for the element in the page layout is the same as if position were static.
- In the example below, note how the other elements are placed as if "Two" were taking up the space of its normal location.



POSITIONING

CSS Relative Position

mainpage.html

nav.html

File Edit View

```
<!DOCTYPE html>
<html lang="en">
<head>
<style>
* {
  box-sizing: border-box;
}

.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#two {
  position: relative;
  top: 20px;
  left: 20px;
  background: blue;
}
</style>
</head>
<body>
<div class="box" id="one">One</div>
<div class="box" id="two">Two</div>
<div class="box" id="three">Three</div>
<div class="box" id="four">Four</div>
</body>
</html>
```

Rayat Prof

WD Theor

New Tab

WD Theor

WD Theor

file:///D:/destop/industrial

One

Two

Three

Four



POSITIONING

CSS Absolute Position

- Positioned concerning its nearest **non-static ancestor**. Elements with position: absolute are taken out of the normal document flow.
- Absolute positioning removes the element from the document flow and places it relative to the nearest ancestor with a positioning context (relative, absolute, or fixed).



POSITIONING

CSS Absolute Position

- An element with `position: absolute;` is positioned relative to the nearest positioned ancestor (with position other than static).
- However; if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.
- Note: Absolute positioned elements are removed from the normal document flow, and can overlap other elements.
- Here is a simple example:

This <div> element has position: relative;

This <div> element has
position: absolute;



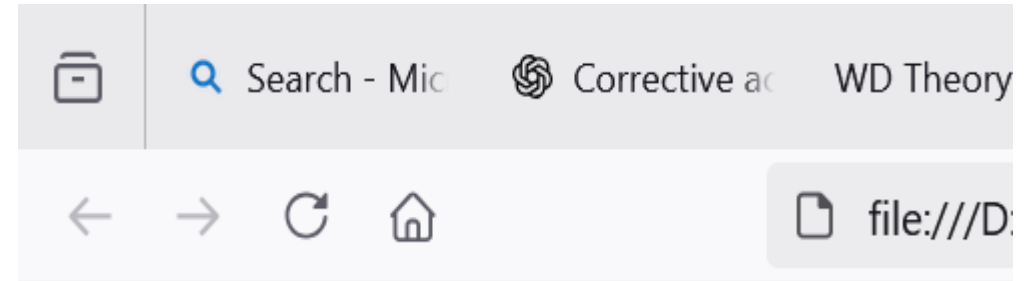
POSITIONING

CSS Absolute Position

```
<!DOCTYPE html>
<html lang="en">
<head>
<style>
* {
  box-sizing: border-box;
}

.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#two {
  position: absolute;
  top: 20px;
  left: 20px;
  background: blue;
}
</style>
</head>
<body>
<div class="box" id="one">One</div>
<div class="box" id="two">Two</div>
<div class="box" id="three">Three</div>
<div class="box" id="four">Four</div>
</body>
</html>
```





POSITIONING

CSS Absolute Position

```
1 body{
2   font-family: tahoma;
3   font-size: 24px;
4 }
5 div.box{
6   border: 5px solid red;
7   width: 600px;
8   height: 600px;
9   overflow: auto;
10  position: relative;
11 }
12
13 div.box img{
14   position: absolute;
15   left: 450px;
16   top: 400px;
17   width: 200px;
18 }
```



Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla





POSITIONING

CSS Sticky Positioning

Sticky positioning switches between relative and fixed based on the scroll position.

An element with `position: sticky;` toggles between a relative and fixed position, depending on the scroll position.

A sticky element is positioned relative until a certain scroll position is reached - then it "sticks" in that place (like `position: fixed`).



POSITIONING

CSS Sticky Positioning

Try to scroll inside this frame to understand how sticky positioning works.

I have position sticky!

Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

I have position sticky!

Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Note: You must specify at least one of the `top`, `right`, `bottom` or `left` properties, for sticky positioning to work.



POSITIONING

CSS Sticky Positioning

Using position: sticky;

Try to scroll this page to understand how sticky works.

Here, when the sticky element reach the top of the page (top: 0), it sticks to this position!

I have position sticky!

Scroll back up to remove the stickiness!

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut

I have position sticky!

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.sticky {
  position: sticky;
  top: 100px;
  padding: 5px;
  background-color: #cae8ca;
  border: 2px solid #4CAF50;
}
</style>
</head>
<body>
```

```
<h2>Using position: sticky;</h2>
```

```
<p>Try to scroll this page to understand how sticky works.</p>
```

```
<p>Here, when the sticky element reach the top of the page (top: 0), it sticks to this position!</p>
```

```
<div class="sticky">I have position sticky!</div>
```

```
<div style="padding-bottom:2000px">
```

```
<p>Scroll back up to remove the stickiness!</p>
```

```
<p>Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.</p>
```

```
<p>Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.</p>
```

```
<p>Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.</p>
```

```
</div>
```



FLOATING

The float property in CSS is **used for positioning and layout, primarily to align an element to the left or right within its containing block, allowing other content (like text) to wrap around it.** This creates a text-wrapping effect similar to what is seen in word processing software.

The CSS float property is used to **specify how an element should float within its parent container.** When an element is floated, it is taken out of the normal document flow and positioned to the left or right, allowing other content to wrap around it. This is particularly useful for creating layouts where text wraps around images or for arranging elements side-by-side.



FLOATING

The float property specifies how an element should float within its container.

It places an element on the left or right side of its container, allowing text and inline elements to wrap around it.

The image inside this <div> is floated to the right.

If you make the screen smaller, you will see that the text wraps around the image.





FLOATING

Values for float:

- none: This is the default value. The element does not float and remains in the normal document flow.
- left: The element floats to the left side of its container, and subsequent content will wrap around its right side.
- right: The element floats to the right side of its container, and subsequent content will wrap around its left side.
- initial: Resets the float property to its default value (none).
- inherit: The element inherits the float value from its parent element.



FLOATING: Right

CSS float: right

In this example, the image will float to the right in the div, and the text will wrap around the image.

Pineapple (\\(Ananas\\) \\(comosus\\)) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking. Pineapple (\\(Ananas\\) \\(comosus\\)) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking.



```
<!DOCTYPE html>
<html>
<head>
<style>
img {
  float: right;
  margin-left: 15px;
  width: 200px;
  height: 200px;
}
</style>
</head>
<body>
```

```
<h2>CSS float: right</h2>
```

```
<p>In this example, the image will float to the right in the div, and the text will wrap around the image.</p>
```

```
<div>

Pineapple (\\(Ananas\\) \\(comosus\\)) is a sweet, juicy tropical fruit that is a good source of vitamins C and B,
fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-
inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm
climates and are used fresh, canned, or in cooking. Pineapple (\\(Ananas\\) \\(comosus\\)) is a sweet, juicy
tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a
digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and
South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking.
</div>
```

```
</body>
</html>
```



FLOATING: Left

CSS float: left

In this example, the image will float to the right in the div, and the text will wrap around the image.



Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking. Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
img {
```

```
  float: left;
```

```
  margin-left: 15px;
```

```
  width: 200px;
```

```
  height: 200px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>CSS float: right</h2>
```

```
<p>In this example, the image will float to the right in the div, and the text will wrap around the image.</p>
```

```
<div>
```

```

```

Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking. Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking.

```
</div>
```

```
</body>
```

```
</html>
```



FLOATING: Left

CSS float: left

In this example, the image will float to the right in the div, and the text will wrap around the image.



Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking. Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
img {
```

```
  float: left;
```

```
  margin-left: 15px;
```

```
  width: 200px;
```

```
  height: 200px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>CSS float: right</h2>
```

```
<p>In this example, the image will float to the right in the div, and the text will wrap around the image.</p>
```

```
<div>
```

```

```

Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking. Pineapple (*Ananas comosus*) is a sweet, juicy tropical fruit that is a good source of vitamins C and B, fiber, potassium, and manganese. It contains a digestive enzyme called bromelain, which can also have anti-inflammatory effects. Native to the Caribbean and South America, pineapples are now grown worldwide in warm climates and are used fresh, canned, or in cooking.

```
</div>
```

```
</body>
```

```
</html>
```



FLOATING: none

```
img {  
  float: none;  
}
```



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio,

vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac...

The float: none value is the default value for float, and the element is displayed just where it occurs in its container.

In the following example the image will be displayed just where it occurs in the container:



ALIGN

- CSS alignment techniques are essential for positioning items and distributing space between content.
- These techniques include horizontal and vertical alignment using various properties such as `margin: auto`, `position: absolute`, `text-align`, and `padding`.



ALIGN

Methods of Aligning in CSS:

1. Center Align Using Margin:

The *margin: auto* property centers a block element horizontally.

Note: Using margin: auto will not work in IE8 unless a !DOCTYPE is declared.

Welcome to RBPU

Center Align Elements

```
<!DOCTYPE html>
<html>

<head>
  <style>
    .center {
      margin: auto;
      width: 60%;
      border: 3px solid #73AD21;
      padding: 10px;
    }
  </style>
</head>

<body>
  <h1 style="color:green;">
    Welcome to RBPU
  </h1>
  <h2>Center Align Elements</h2>
  <div class="center">
    This is div element on which
    margin auto is used to horizontally
    align it into center
  </div>
</body>
```



ALIGN

Methods of Aligning in CSS:

2. Align Using Position:

The *position: absolute* property aligns elements relative to their containing block.

Note: The position properties top, bottom, left, and right will not work unless the position property is set first.

Welcome to RBPU

Right Align

```
<!DOCTYPE html>
<html>

<head>
  <style>
    .right {
      position: absolute;
      right: 100px;
      width: 300px;
      border: 3px solid #73AD21;
      padding: 10px;
    }
  </style>
</head>

<body>
  <h1 style="color:green;">
    Welcome to RBPU
  </h1>
  <h2>Right Align</h2>
  <div class="right">
    <p>
      Absolute positioned elements
      can overlap other elements.
    </p>
  </div>
</body>

</html>
```



ALIGN

Methods of Aligning in CSS:

3. Text Align:

The *text-align: center* property centers text within its container

Example 3: This example describes the CSS align using the *text-align: center;* property.

BOTH TEXTS ARE AT CENTER

This text is centered.

```
<!DOCTYPE html>
<html>

<head>
  <style>
    .center {
      text-align: center;
      border: 2px solid green;
    }
  </style>
</head>

<body>
  <h1 style="color:green;
    text-align: center;">

    </h1>
  <h2>BOTH TEXTS ARE AT CENTER</h2>
  <div class="center">
    <p>This text is centered.</p>
  </div>
</body>

</html>
```



PSEUDO CLASS

- A pseudo-class is a **keyword added to a CSS selector, prefixed by a colon (:), to define a specific state or condition of an element.**
- It is used to style elements like a hovered button, the first child of a container, or checked input fields.
- A CSS pseudo-class **is a keyword added to a selector that defines a special state of an element.**
- This allows developers to style elements based on user interactions, their position within the document structure, or other specific conditions, without altering the HTML markup.



PSEUDO CLASS

Syntax:

Pseudo-classes are indicated by a single colon (:) followed by the pseudo-class name: [🔗](#)

Code

```
selector:pseudo-class-name {  
    /* CSS properties */  
}
```

Syntax

```
selector:pseudo-class {  
    /* styles */  
}
```



PSEUDO CLASS

Common Uses and Examples:

- **User Interaction States:**

- **:hover**: Styles an element when the user's mouse pointer is over it.

Code

```
a:hover {  
    color: blue;  
}
```

- **:active**: Styles an element while it is being activated by the user (e.g., when a mouse button is pressed down on it).

Code



```
button:active {  
    background-color: red;  
}
```



PSEUDO CLASS

Common Uses and Examples:

:focus : Styles an element when it has received focus (e.g., from tabbing with the keyboard).

Code

```
input:focus {  
    border: 2px solid green;  
}
```

- **:visited** / **:link** : Styles visited and unvisited links differently based on browser history.

Code

```
a:link {  
    color: purple;  
}  
a:visited {  
    color: gray;  
}
```



PSEUDO CLASS

- **Structural and Positional States:**

- `:first-child`: Styles an element that is the first child of its parent. 

Code



```
li:first-child {  
    font-weight: bold;  
}
```

- `:last-child`: Styles an element that is the last child of its parent.
- `:nth-child(n)`: Styles an element based on its position among its siblings (e.g., `:nth-child(odd)` or `:nth-child(2n)`).
- `:root`: Targets the highest-level element in the document (usually `<html>`). 
- **Form Element States:**
- `:checked`: Styles a checked radio button or checkbox.
- `:required` / `:optional`: Styles form elements based on the presence of the `required` attribute.
- `:valid` / `:invalid`: Styles form elements based on whether their content is valid according to their constraints.

PSEUDO CLASS EXAMPLE

```
<html>
<head>
<style>
a:visited {color: blue;}
a:hover {color: yellow;}
a:active {color: orange;}
</style>
</head>
<body>
<H1>
<a href="abc.html ">Click Here</a></H1>
</body>
</html>
```



Image Sprites

- CSS Image Sprites are a technique used **to optimize web page loading by combining multiple small images, such as icons or buttons, into a single larger image file. This combined image is known as a "sprite image."**
- The primary benefit of using image sprites is the **reduction of HTTP requests a browser makes to the server.**
- When a web page contains many individual images, each image requires a separate HTTP request, which can slow down page loading time. By consolidating these images into a single sprite, only one request is needed to download all of them, significantly improving performance.



Image Sprites

CSS Image Sprites

An image sprite is a collection of various small images put into one larger image file, called a "sprite image".

A sprite image is typically arranged in a grid-like way, like this:



A web page with multiple images takes a longer time to load, and generates multiple server requests.

So, instead of downloading each image separately, the browser downloads the single sprite image file, which will reduce the number of server requests and reduce bandwidth usage.



Image Sprites

How CSS Image Sprites Work:

•Create a Sprite Image:

Combine all the desired small images into a single, larger image file. Tools are available online to assist with this process.

•Define CSS Styles:

- Set the **background-image** property of the element to the URL of the sprite image.
- Define the **width and height** of the element to match the dimensions of the individual image within the sprite that you want to display.
- Use the **background-position** property to specify the exact coordinates (x and y values) within the sprite image where the desired individual image begins. This effectively "crops" and displays only the relevant portion of the sprite image within the element's defined width and height.



Image Sprites

The CSS key properties used for image sprites are:

- background-image
- background-position



Image Sprites

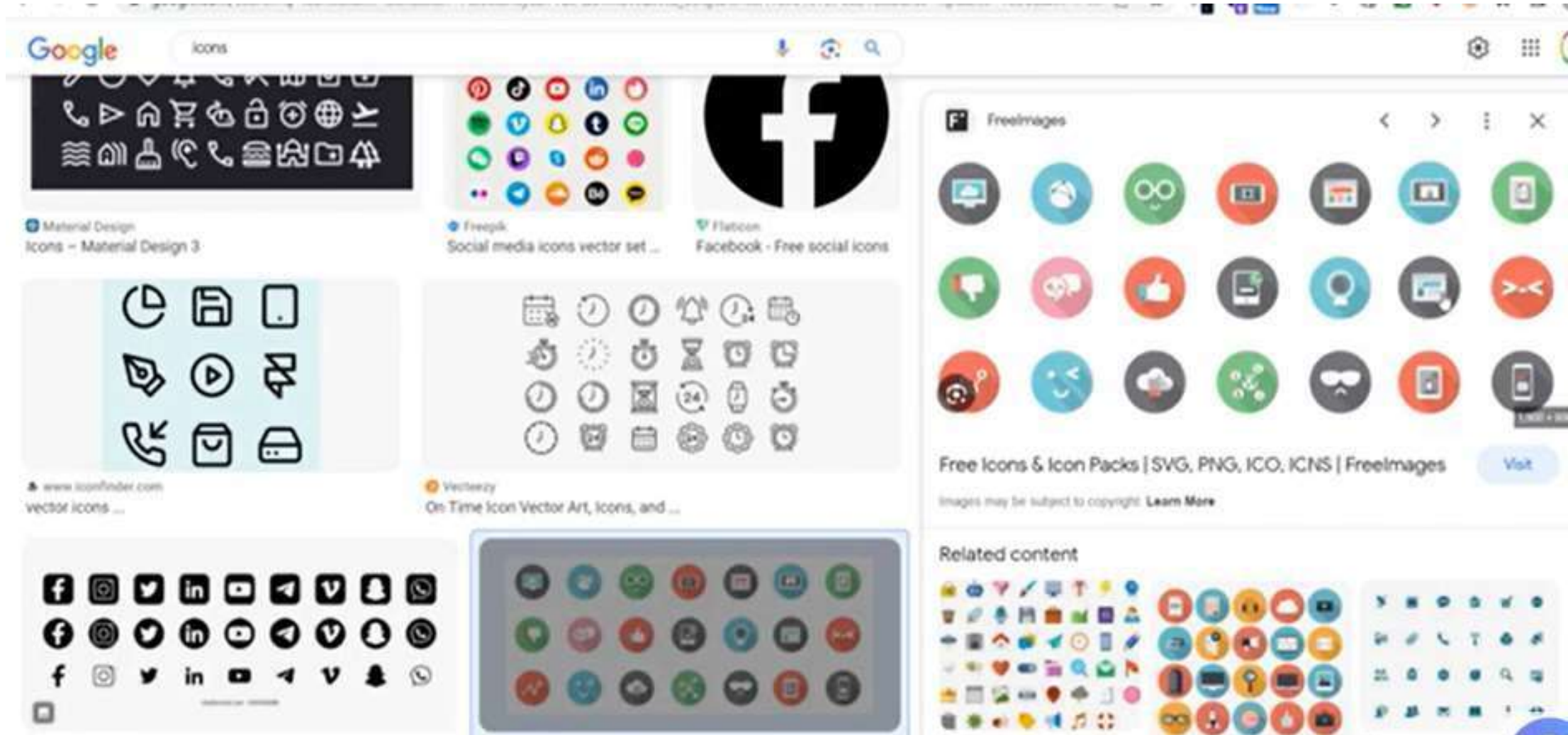




Image Sprites

```
<!DOCTYPE html>
<html>
<head>
<style>
#home {
  width: 46px;
  height: 44px;
  background-image: url(img_navsprites.gif);
  background-position: 0 0;
}

#prev {
  width: 43px;
  height: 44px;
  background-image: url('img_navsprites.gif');
  background-position: -47px 0;
}

#next {
  width: 43px;
  height: 44px;
  background-image: url('img_navsprites.gif');
  background-position: -91px 0;
}
</style>
</head>
<body>

<h2>Image Sprites</h2>


<br>
<br>

</body>
</html>
```





Image Sprites

Advantages:

- **Faster Page Load Times:** Reduces the number of HTTP requests, leading to quicker rendering of the page.
- **Reduced Bandwidth Usage:** Sending one larger file can sometimes be more efficient than sending many small files.
- **Improved User Experience:** A more responsive website due to faster loading.



Attribute sector

CSS attribute Selector allows you to select elements based on the presence, value, or specific characteristics of their attributes. They are particularly useful for dynamic or structured content where attributes play a key role, such as in forms or data tables.

Types of CSS Attribute Selectors

1. [attribute] Selector

This selector matches all elements with the specified attribute, regardless of its value.

```
<style>
  /* Styles all elements with a 'placeholder' attribute */
  [placeholder] {
    border: 2px solid blue;
  }
</style>
```



Attribute sector

Types of CSS Attribute Selectors

```
<html>
<head>

  <style>
    /* Styles all elements with a 'placeholder' attribute */
    [placeholder] {
      border: 2px solid blue;
    }
  </style>

</head>
<body>
  <input type="text" placeholder="Enter your name">
  <input type="password" placeholder="Enter your password">
</body>
</html>
```



Attribute sector

Types of CSS Attribute Selectors

2. [attribute="value"] Selector

This selector matches elements with an attribute whose value is exactly equal to the specified value.

```
<html>
<head>

  <style>
    /* Styles the button with type="submit" */
    [type="submit"] {
      background-color: green;
      color: white;
    }
  </style>

</head>
<body>
  <button type="submit">Submit</button>
  <button type="reset">Reset</button>
</body>
</html>
```





Attribute sector

Types of CSS Attribute Selectors

3. [attribute~="value"] Selector

This selector matches elements where the attribute's value is a space-separated list of words and one of the words matches the specified value.



```
<html>
<head>

  <style>
    /* Styles elements where class includes 'green' */
    [class~="green"] {
      border: 3px solid red;
    }
    .box {
      height: 100px;
      width: 100px;
      position: relative;
      background-color: green;
    }
    .container {
      display: flex;
      height: 500px;
      width: 100vw;
      gap: 30px;
    }
  </style>

</head>
<body>
  <div class="container">
    <div class="box large green"></div>
    <div class="box small red"></div>
  </div>
</body>
</html>
```



Attribute sector

Types of CSS Attribute Selectors

4. [attribute]="value" Selector

This selector matches elements whose attribute value is either exactly value or starts with value followed by a hyphen (-). It is often used for language attributes.

```
<html>
<head>

  <style>
    /* Styles elements with lang="en-US" or lang="en-GB" */
    [lang]="en" {
      color: blue;
    }
  </style>

</head>
<body>
  <p lang="en-US">Hello</p>
  <p lang="en-GB">Hello</p>
  <p lang="fr">Bonjour</p>
</body>
</html>
```

Hello

Hello

Bonjour



Attribute sector

Types of CSS Attribute Selectors

5. [attribute^="value"] Selector

This selector matches elements whose attribute value starts with the specified value.

```
<html>
<head>

  <style>
    /* Styles links with href starting with 'https' */
    [href^="https"] {
      text-decoration: underline;
      color: darkgreen;
    }
  </style>

</head>
<body>
  <a href="https://example.com/">Visit Example</a>
  <br>
  <a href="mailto:contact@example.com">Email Us</a>
</body>
</html>
```

[Visit Example](https://example.com/)
[Email Us](mailto:contact@example.com)



Attribute sector

Types of CSS Attribute Selectors

6. [attribute\$="value"] Selector

This selector matches elements whose attribute value ends with the specified value

```
<html>
<head>

  <style>
    /* Selects all elements whose class ends with "box" */
    [class$="box"] {
      border: 2px solid blue;
      padding: 10px;
      background-color: lightgray;
    }
  </style>

</head>
<body>
  <div class="small-box">This is a small box</div>
  <div class="large-box">This is a large box</div>
  <div class="circle">This is not a box</div>
</body>
</html>
```

This is a small box

This is a large box

This is not a box



Attribute sector

Types of CSS Attribute Selectors

7. [attribute="value"] Selector*

This selector matches elements whose attribute value contains the specified value anywhere within it.

```
<html>
<head>

  <style>
    /* selects all div elements where class contains "box" */
    div[class*="box"] {
      background-color: lightgreen;
      color: darkgreen;
      padding: 20px;
      border: 2px solid green;
    }
  </style>

</head>
<body>
  <div class="small-box">This is a small box</div>
  <div class="large-box">This is a large box</div>
  <div class="circle">This is not a box</div>
</body>
</html>
```

This is a small box

This is a large box

This is not a box



Attribute sector

Types of CSS Attribute Selectors

8. Combining Attribute Selectors

You can combine multiple attribute selectors to refine your targeting.

```
<html>
<head>

  <style>
    /* Styles inputs that are both type="text" and data-required="true" */
    [type="text"][data-required="true"] {
      background-color: green;
    }
  </style>

</head>
<body>
  <input type="text" data-required="true">
  <input type="password" data-required="true">
</body>
</html>
```





Attribute sector

Types of CSS Attribute Selectors

8. Combining Attribute Selectors

You can combine multiple attribute selectors to refine your targeting.

```
<html>
<head>

  <style>
    /* styles inputs that are both type="text" and data-required="true" */
    [type="text"][data-required="true"] {
      background-color: green;
    }
  </style>

</head>
<body>
  <input type="text" data-required="true">
  <input type="password" data-required="true">
</body>
</html>
```



CSS COLOR

- CSS provides various ways to **specify colors for styling web elements**. These methods allow for flexibility in defining colors for text, backgrounds, borders, and more.
- CSS colors are used to change the look of text, backgrounds, borders, and other elements on a webpage. They help make a site more attractive and easy to read.
- Colors can be set using names, HEX codes, RGB, RGBA, HSL, or HSLA values.
- Used to style text, backgrounds, and borders.
- Help create contrast, highlight content, and improve visual design.



CSS COLOR

CSS provides various ways to specify colors for styling web elements. These methods allow for flexibility in defining colors for text, backgrounds, borders, and more.

Methods for Specifying Colors in CSS: Predefined Color Names.

CSS supports 140 standard color names, such as red, blue, green, orange, purple, gray, etc.

Code

```
p {  
    color: blue;  
}
```



CSS COLOR

Hexadecimal (Hex) Values.

Hex codes represent colors using a six-digit hexadecimal number, prefixed with a #. Each pair of digits represents the intensity of red, green, and blue, respectively (from 00 to FF).

Code

```
body {  
    background-color: #FF0000; /* Red */  
}
```



CSS COLOR

RGB Values.

RGB values define colors by specifying the intensity of red, green, and blue light (from 0 to 255).

Code

```
h1 {  
    color: rgb(0, 128, 0); /* Green */  
}
```



CSS COLOR

RGBA Values.

RGBA extends RGB by adding an alpha channel for opacity (from 0.0 to 1.0).

Code

```
div {  
    background-color: rgba(255, 0, 0, 0.5); /* Semi-transparent red */  
}
```



CSS COLOR

RGBA Values.

RGBA extends RGB by adding an alpha channel for opacity (from 0.0 to 1.0).

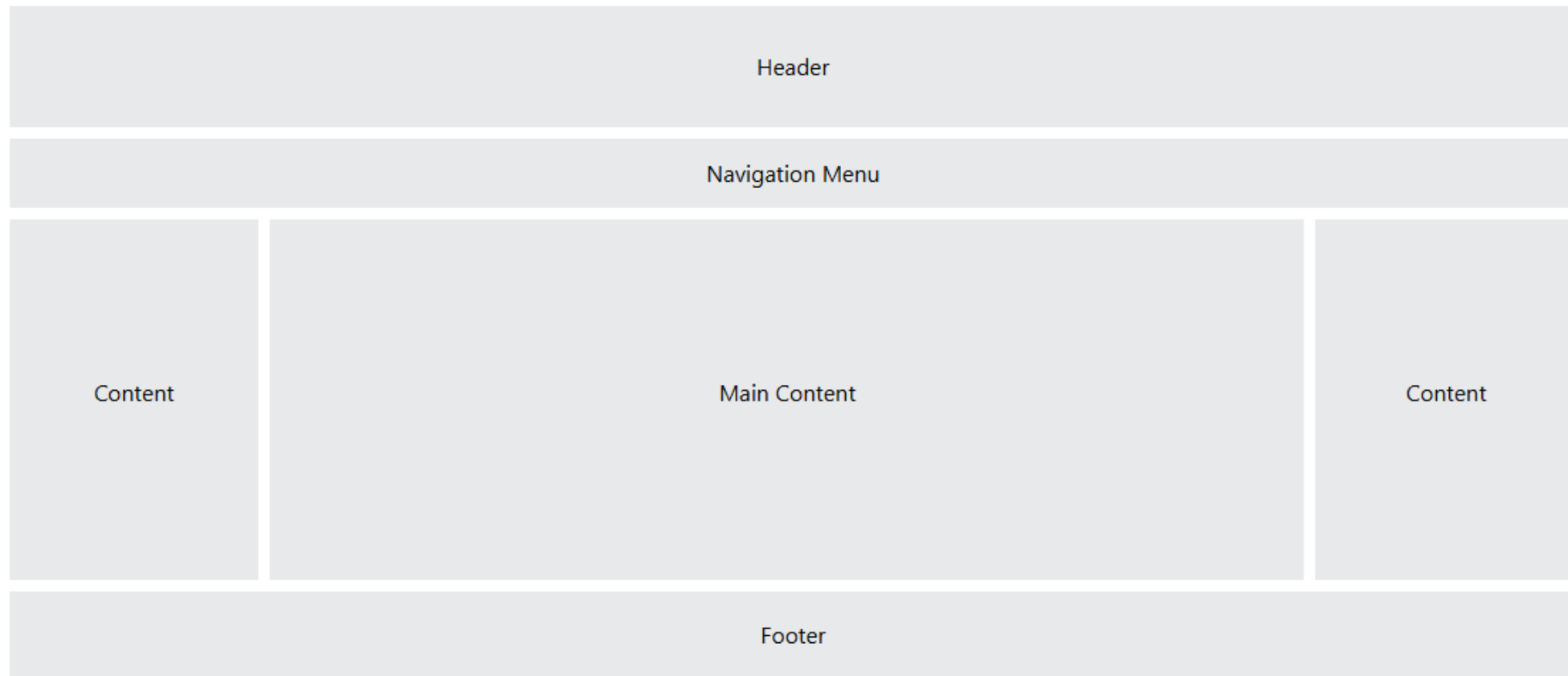
Code

```
div {  
    background-color: rgba(255, 0, 0, 0.5); /* Semi-transparent red */  
}
```



Creating page Layout and Site Designs.

A website is often divided into multiple sections, like a top header, navigation menu, main content, and a footer:



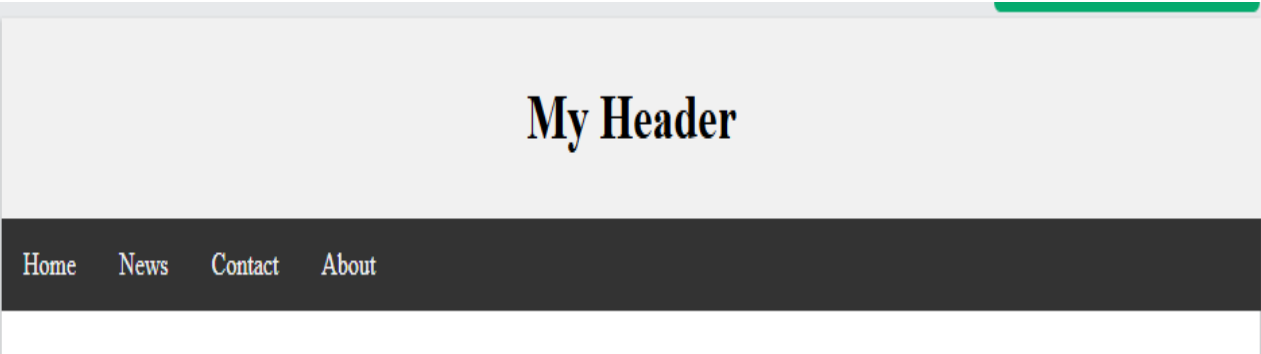
There are tons of different layout designs to choose from. However, the structure above, is one of the most common, and we will take a closer look at it in this tutorial.



Creating page Layout and Site Designs.

CSS Navigation Bar

A navigation bar contains a list of links to help visitors navigate through your website:



My Header

[Home](#) [News](#) [Contact](#) [About](#)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  padding: 0;
}

header {
  background-color: #f1f1f1;
  padding: 10px;
  text-align: center;
}

/* Style the topnav */
ul.topnav {
  display: flex;
  list-style-type: none;
  margin: 0;
  padding: 0;
  background-color: #333333;
}

/* Style links in topnav */
ul.topnav li a {
  display: block;
  color: #f1f1f1;
  padding: 14px 16px;
  text-decoration: none;
}
```



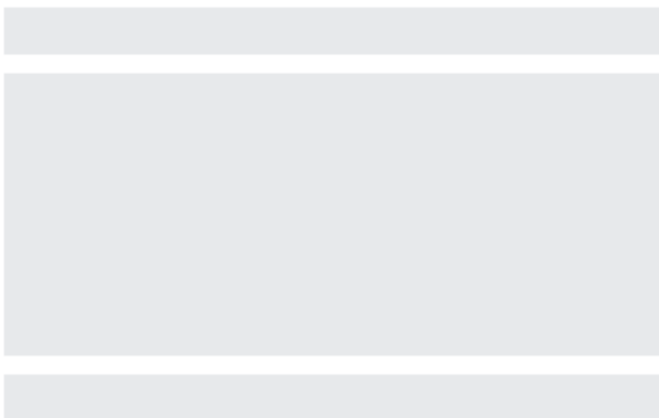
Creating page Layout and Site Designs.

CSS Layout Content

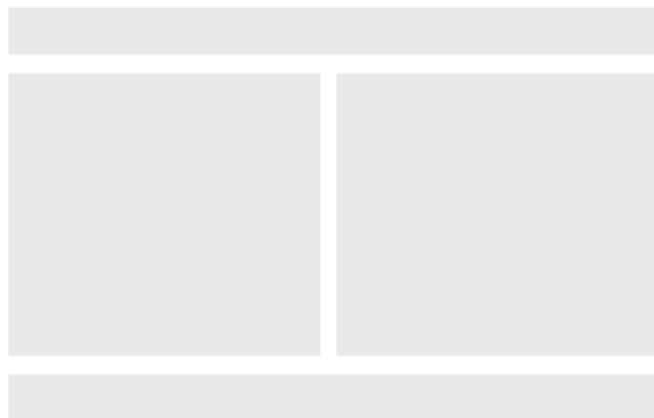
How the content of a website should be shown, often depends on the device of the users. The most common layouts are:

- **1-column layout** (often used for mobile browsers)
- **2-columns layout** (often used for tablets and laptops)
- **3-columns layout** (only used for desktops)

1-column:



2-column:



3-column:





Creating page Layout and Site Designs.

Example

```
div.flex-container {  
  display: flex;  
  /* Show the flex items horizontally */  
  flex-direction: row;  
}  
  
div.flex-container > div {  
  margin: 10px;  
}  
  
/* Use media query and show the flex items vertically if screen width is less than 600px */  
@media screen and (max-width:600px) {  
  div.flex-container {  
    flex-direction: column;  
  }  
}
```

Column

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas sit amet pretium urna. Vivamus venenatis velit nec neque ultricies, eget elementum magna tristique.

Column

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas sit amet pretium urna. Vivamus venenatis velit nec neque ultricies, eget elementum magna tristique.

Column

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas sit amet pretium urna. Vivamus venenatis velit nec neque ultricies, eget elementum magna tristique.



Creating page Layout and Site Designs.

CSS Basic and Fixed Footer

The footer is placed at the bottom of a webpage. It often contains information like copyright and contact info.

The following example shows a basic footer styling:

Example

```
footer {  
  background-color: #f1f1f1;  
  text-align: center;  
  padding: 8px;  
}
```

Result

Footer